Computability: Recitation 7

12.05.2025

1 Non-Deterministic Turing Machines

Definition 1 (Non-Deterministic Turing Machine (NTM))**.** *A* non-deterministic Turing machine *is a 7-tuple*

$$M = \langle \Sigma, \Gamma, \sqcup, Q, q_0, F, \delta \rangle$$

where:

- Σ *is a finite input alphabet.*
- Γ *is a finite tape alphabet, with* $\Sigma \subseteq \Gamma$*.*
- $\sqcup \in \Gamma \setminus \Sigma$ *is the blank symbol.*
- Q *is a finite, non-empty set of states.*
- $Q_0 \subseteq Q$ *is set of initial states.*
- $F \subseteq Q$ *is the set of final states.*
- $\delta : (Q \setminus F) \times \Gamma \to 2^{Q \times \Gamma \times \{L,R\}}$ *is the transition function.*

The transition function δ *maps a state and a tape symbol to a set of possible actions, where each action consists of a next state, a symbol to write, and a direction to move the tape head (left or right).*

Note: An NTM may have multiple runs on a given input word w.

Definition 2 (Acceptance)**.** *Let* N *be an NTM and let* $w \in \Sigma^*$*. We say that* N accepts w *if there exists at least one accepting run of* N *on* w *- that is, a run that ends in* q_{acc}*. In this case, we write* $w \in L(N)$*.*

Example 3 (CLIQUE)**.** *Let* $G = (V, E)$ *be a graph. A* clique *is a set* $S \subseteq V$ *such that there is an edge between every pair of vertices in* S*. Define the following decision problem:*

$$CLIQUE = \{\langle G, k \rangle : \text{There is a clique of size } k \text{ in } G\}$$

We describe a nondeterministic Turing machine (NTM) that decides CLIQUE*:*

- *The machine nondeterministically writes on the tape the names of* k *distinct vertices from* V*, separated by #. This guessing is done by replacing a blank section of the tape with symbols from the encoding of vertex names, one at a time, using nondeterministic transitions.*
- *Then, the machine checks deterministically whether all* $\binom{k}{2}$ *pairs among the chosen vertices are connected by edges in* G*. For each pair, it searches through the encoding of* E *to see if the corresponding edge exists.*
- *If all pairs are connected, the machine accepts. Otherwise, it rejects.*

This machine runs in nondeterministic polynomial time:

- *Guessing the* k *vertex names takes polynomial time, since* $k \leq |V|$*.*
- *Checking all* $\binom{k}{2}$ *pairs for the existence of edges can also be done in polynomial time, by scanning the input encoding of* G*.*

Therefore, CLIQUE *can be solved by a nondeterministic Turing machine in polynomial time.*

Remark: *A* trivial *deterministic algorithm would try all subsets of k vertices and check if any of them forms a clique. However, the number of such subsets is* $\binom{|V|}{k}$*, which is exponential in the input size when k is not a constant because:*

$$\binom{|V|}{|V|/2} \approx 2^{|V|}$$

Since each check takes polynomial time but must be repeated exponentially many times, this yields an exponential-time algorithm overall. Thus, the trivial approach does not give a polynomial-time solution.

2 P and NP

Definition 4 (Runtime for TM)**.** *The* runtime *of a TM M on input* $w \in \Sigma^*$ *is the number of steps M performs until it halts.*

Definition 5 (Runtime for NTM)**.** *The* runtime *of a NTM N on input* $w \in \Sigma^*$ *is the number of steps N performs until it halts in the longest run on w.*

$$runtime_N(w) = max\{|r| : r \text{ is a run of } N \text{ on } w\}$$

Remark: This value can be ∞ in case that there is a run of N that forms infinite loop.

Remark. One may alternatively define the running time of a nondeterministic Turing machine using the *shortest accepting run*, rather than the longest run.

The idea is the following. Since some computation runs may loop forever, or may continue for unnecessarily many steps even when a short accepting computation exists, we modify the machine so that it first nondeterministically guesses a natural number t, thought of as a "timer". The machine then simulates the original computation for at most t steps. If the simulation has not accepted within those t steps, the run halts and rejects.

With this modification:

- If the original machine has an accepting computation run of length k, then there exists a run that guesses $t \geq k$ and accepts.
- Every computation run halts after at most t simulated steps, so both infinite loops and unnecessarily long computations are cut off.

From this point of view, the running time of a nondeterministic machine can be defined as the length of the shortest accepting computation run or the longest t we choose not deterministically.

Definition 6 (Runtime Complexity)**.** *For* $f : \mathbb{N} \to \mathbb{N}$*, we say that a TM/NTM T runs in time* $O(f(n))$ *if:*

$$\forall w \in \Sigma^* \quad runtime_T(w) = O(f(|w|))$$

Definition 7 (Time/NTime)**.** *For* $f : \mathbb{N} \to \mathbb{N}$ *we define:*

$$Time(f(n)) = \{L \subseteq \Sigma^* | L \text{ can be decided by a TM that runs in } O(f(n)) \text{ time}\}$$

$$NTime(f(n)) = \{L \subseteq \Sigma^* | L \text{ can be decided by a NTM that runs in } O(f(n)) \text{ time}\}$$

Definition 8 (P/NP)**.** *The class of languages that can be decided in polynomial time by TM/NTM*

$$\mathrm{P} = \bigcup_{k=1}^{\infty} Time(n^k)$$

$$\mathrm{NP} = \bigcup_{k=1}^{\infty} NTime(n^k)$$

Definition 9 (EXP)**.** *The class of languages that can be decided in exponential time by TM*

$$\mathrm{EXP} = \bigcup_{k=1}^{\infty} Time(2^{n^k})$$

Remark: If a language can be decided by a TM that runs in polynomial or exponential time, then in particular it is decidable. Therefore it follows easily from the definition that $\mathrm{P}, EXP \subseteq \mathrm{R}$.

Moreover, in the exercise you will show that $NP \subseteq EXP$, (by describing a TM that simulate all possible runs of the NTM) thus also $NP \subseteq R$.

2.1 Closure Properties for P and NP

Proposition 10. P *is closed under union.*

Proof. Let $L_1,\ L_2 \in \mathrm{P}$. Then there are TMs M_1, M_2 that decide L_1, L_2 respectively, and run in polynomial time. Define a new TM M which operates as follows on input $x \in \Sigma^*$:

- Run M_1 on x. If it accepts, accept.
- Run M_2 on x. If it accepts, accept.
- If neither accepted, reject.

The proof that $L(M) = L_1 \cup L_2$ is identical to the case of union in R (shown in a previous recitation). The runtime of M_1 and M_2 is polynomial, so the sum is also polynomial.

Remark: during the run of M_1 on x, the machine M must keep a copy of x for later (to clean up the tape for M_2). This involves overhead: after each step that M_1 performs, M may have to move the copy. The cost is polynomial in $|x|$, per step. So the overall runtime of M is also polynomial. □

Remark: In the proof above, we construct from M_1, M_2 a new TM that "contains" them. This is different from a universal TM that simulates a machine given its encoding on the tape. Notice that such a simulation is also polynomial: you have seen in exercise 5 that one can implement a universal TM U such that simulating each step of a TM M takes polynomial time.

Proposition 11. NP *is closed under union.*

Proof. Let $L_1, L_2 \in \mathrm{NP}$. Then there exist non-deterministic polynomial-time Turing machines N_1 and N_2 that decide L_1 and L_2, respectively.

We construct a new non-deterministic Turing machine N that decides $L_1 \cup L_2$ as follows:

1. On input w, N non-deterministically chooses whether to simulate N_1 or N_2.
2. It then simulates the chosen machine on input w.
3. If the simulation accepts, then N accepts. Otherwise, it rejects.

Since both N_1 and N_2 run in polynomial time, and the non-deterministic choice adds only a constant overhead, the machine N also runs in non-deterministic polynomial time (The maximum of the time of N_1 and N_2).

Moreover, N accepts w if and only if $w \in L_1$ or $w \in L_2$, so $L(N) = L_1 \cup L_2 \in \mathrm{NP}$. □

Proposition 12. P *is closed under complement.*

Proof. Let $L \in \mathsf{P}$, and let M be a deterministic Turing machine that decides L in polynomial time. Construct a machine M' by swapping the accepting and rejecting states of M. Since this transformation only modifies the final states and leaves the computation unchanged, M' still runs in polynomial time and decides the complement language $\overline{L}$. Hence, $\overline{L} \in \mathsf{P}$. □

Remark. This argument does not extend to NP. For example, consider the language CLIQUE, which we have already seen is in NP.

Now consider the complement language:

$$\overline{\mathrm{CLIQUE}} = \{\langle G, k\rangle : \text{Every size-}k \text{ subset of vertices is not a clique}\}$$

To decide this language, we must confirm that *no* subset of size k forms a clique. A nondeterministic machine cannot verify this efficiently: it can guess a subset and reject if it is a clique, but if it guesses a non-clique, that tells us nothing about the existence of a different subset that might be a clique.

Thus, flipping accept and reject states of the nondeterministic machine for CLIQUE does not yield a valid machine for $\overline{\text{CLIQUE}}$, since rejection does not mean "no" in the presence of nondeterminism.

In fact, it is currently unknown whether NP is closed under complement; that is, whether $\mathsf{NP} = \mathsf{coNP}$ remains an open question.

3 Graph Problems

Graph encoding: In the following examples we use graphs, so we must encode them as words in some way, as we did for TMs. For a graph $G = (V, E)$, we denote by $\langle G \rangle$ its encoding as a word over $\{0, 1, \#\}$. It contains a binary name for each vertex in V, and a pair of such names for each edge in E. Notice that this encoding is the same size as the graph $|V| + |E|$ (up to a polynomial).

Example: Let $L = \{\langle G \rangle : G \text{ is a connected graph}\}$. We show that $L \in \text{P}$. Recall that in previous courses, we have done this using graph search (such as DFS):

- Choose an arbitrary vertex $v_0 \in V$.
- Perform DFS starting in v_0.
- If all vertices have been visited, the graph is connected. Otherwise, it is not.

The time complexity we learned was $O(|V| + |E|)$ for DFS. Does this still hold for TMs? No: in every iteration of DFS, just checking whether the vertex has been visited before, and finding its neighbors, requires scanning the tape several times. The TM is not as efficient. But, importantly, the difference in efficiency is *polynomial*. This is because the main difference between TMs and the model we are used to (random access memory) is that the TM cannot "jump" to a memory cell, so we pay a (polynomial) price in time per step.

Note: There are many important decision problems on graphs. For example:

- An *independent set* in a graph $G = (V, E)$ is a subset $S \subseteq V$ such that no two vertices in S are connected by an edge.
- A *Hamiltonian path* is a path that visits every vertex in the graph *exactly once*.

We define the following decision problems:

$$\text{IS} = \{\langle G, k \rangle : \text{There is an independent set of size } k \text{ in } G\}$$
$$\text{HAM-PATH} = \{\langle G \rangle : \text{There is a Hamiltonian path in } G\}$$

A natural question arises: *which of these problems are in P, and which are in* NP*?* How can we formally compare the complexity of such problems?

To answer this, we introduce the notion of *polynomial-time reductions*, which help us classify problems and understand the relationships between them within the complexity hierarchy.

4 Polynomial reductions

Definition 13 (Computable in polynomial time)**.** *A function $f : \Sigma^* \to \Sigma^*$ is* computable in polynomial time *if there exists a TM M_f that runs in polynomial time, and for every input $x \in \Sigma^*$, when M_f halts, the word $f(x)$ is written on the tape.*

Definition 14 (polynomial reduction)**.** *A* polynomial reduction *from a language $A \subseteq \Sigma^*$ to a language $B \subseteq \Sigma^*$ is a reduction $f : \Sigma^* \to \Sigma^*$ from A to B that is computable in polynomial time. If such a reduction exists, we denote $A \leq_p B$.*

Motivation: We have seen in mapping reductions that if we can convert a problem to another type of problem which we can solve, then we can solve the original problem. We would like to say a similar thing about P, but it requires polynomial reductions (why?).

Theorem 15. *The reduction theorem for* P*: If $A \leq_p B$ and $B \in \text{P}$, then $A \in \text{P}$.*

Proof. Suppose $A \leq_p B$. There is a TM M_f that computes the reduction in polynomial time. Since $B \in \text{P}$, there is some TM M_B that decides B in polynomial time. We define M_A which runs as follows on input $x \in \Sigma^*$:

- Run M_f on x, to obtain $f(x)$.
- Run M_B on $f(x)$, and accept or reject accordingly.

Correctness is the same as the reduction theorem for mapping reductions. It remains to prove that this algorithm indeed runs in polynomial time.

The output of $M_f(x)$ is of length $O(|x|^k)$ for some constant k, and the runtime of M_B on an input of size $O(|x|^k)$ is $O((|x|^k)^m)$ for some constant m. Therefore the running time is $O(|x|^{mk})$, which is polynomial in $|x|$. □

Example: In a previous recitation, we looked at the languages:

$$L_0 = \{x : |x| \mod 2 = 0\}, \quad L_1 = \{x : |x| \mod 2 = 1\}$$

We showed that $L_0 \leq_m L_1$ by a reduction that adds a letter: $f(w) = aw$. This reduction can be implemented in polynomial time (think how), so we have $L_0 \leq_p L_1$.

Remark: Go over the reductions we have seen so far, and check which are polynomial (many of them are).

Proposition 16. *Recall the languages:*

$$\text{CLIQUE} = \{\langle G, k\rangle : \textit{There is a clique of size } k \textit{ in } G\}$$
$$\text{IS} = \{\langle G, k\rangle : \textit{There is an independent set of size } k \textit{ in } G\}$$

We have CLIQUE $\leq_p$ IS.

Proof. The idea is that cliques and independent sets are complementary: we can convert between them by complementing the graph.

- **Construction:** Let $f(\langle G, k\rangle) = \langle \overline{G}, k\rangle$, where $\overline{G}$ is obtained from G by removing the edges, and adding edges between vertices that were not connected before. Formally, $\overline{G} = (V, \overline{E})$ where:

$$\overline{E} = \{\{u, v\} : u, v \text{ are distinct vertices and } \{u, v\} \notin E\}$$

- **Correctness:**
  - Suppose $\langle G, k\rangle \in$ CLIQUE. Then there is some set of vertices S of size k such that there is an edge between every two vertices in S. By definition of $\overline{G}$, there are no edges at all between the vertices in S in $\overline{G}$, so it is an independent set of size k and we have $\langle \overline{G}, k\rangle \in$ IS.
  - Suppose $\langle \overline{G}, k\rangle \in$ IS. Then there exists a set $S \subseteq V$ of size k such that there are no edges between any two vertices in S. Thus S is a clique in G, so $\langle G, k\rangle \in$ CLIQUE
- **Complexity:** A TM can iterate over every $\{u, v\} \in V \times V$, check if it's in E and add or not add it to $\overline{G}$ accordingly. There are $O(|V|^2)$ pairs to check, and each takes polynomial time (we iterate over the edges of G and perform string comparisons). Thus the reduction can be computed in polynomial time.

□

Remark: We don't know whether CLIQUE and IS are in P or not. However now we know that if IS $\in$ P then CLIQUE $\in$ P. There is also a polynomial reduction in the other direction (see exercise 7). So they "go together": either both are in P or both are not in P. We will see that this situation applies to many other problems.

5 An exponential algorithm (if time permits)

Proposition 17. *We have* CLIQUE $\in$ EXP.

Proof. The following TM decides CLIQUE in exponential time:

- For every $C \subseteq V$ such that $|C| = k$:
  - If C is a clique, accept.
- Reject.

Checking whether a given subset C is a clique takes polynomial time (we need to check whether there is an edge between each of $\frac{k(k-1)}{2} = O(k^2)$ pairs of vertices). There are $\binom{|V|}{k} = O(|V|^k) = O(2^{k \log |V|})$ subsets of size k, which is exponential. □

Remarks:

- The number k is not a constant but a part of the input, so $|V|^k$ is not polynomial.
- The binomial coefficient $\binom{|V|}{k}$ is slightly smaller than $|V|^k$, but not significantly for this calculation.
- The algorithm is exponential, so we managed to classify CLIQUE into the class EXP. This is only an *upper bound*: it does not tell us whether CLIQUE is also in P or not.